Small Universal Reversible Turing Machines Constructed by AI and Formally Verified in Lean without Human Inspection of the Formalization

Tetsuo Yokoyama
Nanzan University

October 5, 2026
Extended version

Abstract

The AI proved two main results on small universal reversible Turing machines (URTMs) using a theorem prover. First, it constructed seven URTM(m, n), each with m states and n symbols, that are not dominated by any of the nine URTMs listed in Morita's book (2017). They are $(m, n) = (3, 600)$, $(4, 42)$, $(5, 39)$, $(6, 30)$, $(7, 28)$, $(8, 21)$, and $(85, 2)$. Among these seven points and Morita's nine points, 13 are non-dominated. For all 13 points, the AI proved that concrete rule tables are deterministic and reversible. It also proved that they are universal under the formalization of Morita's definition used here, which includes computable encoding and decoding. Second, the AI showed that the halting problem of reversible Turing machines with at most two states is decidable for finite instantaneous descriptions. As a consequence, it proved that no URTM with at most two states exists under this formalization. It also proved that no three-state, two-symbol RTM is universal under this formalization. We obtained these results under the following protocol. The human gives the problem and the direction. The AI writes the formalization and the proofs for the theorem prover. The human inspects neither the formalization nor the proofs. We count as results only theorems that the theorem prover accepts, that contain no proof placeholders, and that depend only on the standard axioms. Of the seven points, $(85, 2)$ was found by an automated search in which the human did not intervene after starting it. Whether the formal definitions and theorem statements express the intended claims has not been checked mechanically or by the human. If the formalization contains an error, some or all of the claims of this note may fail.

1 Introduction

Reversible computing concerns computations in which each step uniquely determines its predecessor, if any. Bennett showed that ordinary irreversible computations can be simulated reversibly without loss of computational power [1]. A reversible Turing machine (RTM) is one of the basic models of reversible computing. A universal RTM is called a universal reversible Turing machine (URTM). We measure the size of a URTM by its numbers of states and symbols. We write URTM(m, n) for a URTM with m states and n symbols.

Morita's book [2] lists nine known URTMs. The book states that the three of them obtained by conversion, $(4, 168)$, $(138, 2)$, and $(3, 36654)$, leave room for reduction. It also states that it is open whether a two-state URTM exists. In a 2023 survey, after stating that the class of two-state TMs is universal in the irreversible case, Morita writes, "However, it is unknown whether the class of two-state RTMs is universal" [3].

An advantage of a proof assistant is that a small kernel decides whether a proof is correct [4]. However, definitions and theorem statements written by an AI may not express the intended mathematics, even when Lean accepts them [6]. The AI performed the search, the formalization, and the proofs. The human did not inspect the formalization or the proofs.

The contributions of this note are as follows.

- The AI constructed seven URTMs that are not dominated by any of Morita's nine points. Among these seven points and Morita's nine points, 13 are non-dominated. Their universality under the Lean formalization of Morita's definition used here was also proved in Lean (Section 3.2).
- The AI showed that the halting problem of RTMs with at most two states is decidable for finite instantaneous descriptions (IDs), and proved in Lean that no URTM with at most two states exists under this formalization (Section 3.4).
- For three-state, two-symbol RTMs, the AI proved in Lean that none is universal under this formalization (Section 3.5).

For each state count, our construction gives an upper bound on the minimum symbol count for a URTM.

Section 2 explains what this protocol guarantees and what it does not.

2 Methods

2.1 Protocol

We worked under the following protocol.

(P1) *Goal.* Obtain proofs in Lean that contain no `sorry`.

(P2) *Acceptance condition.* A theorem is regarded as a result only if Lean accepts it and its axiom list contains only the standard axioms. Lean outputs this list. This condition is checked mechanically, so it needs no human judgment.

(P3) *Formalization by the AI.* The AI also writes the definitions in Lean, such as RTMs, reversibility, simulation, and universality. The human does not directly inspect whether the definitions faithfully express the original source.

(P4) *Role of the human.* The human only gives the problem, decides the direction and the order of work, allocates the usage limits of the AI and the computers, and selects the results. This includes choosing which AI checks the work and when.

"Does not directly inspect" in (P3) means the following. The human does not read the Lean code, the definitions, or the proofs in order to judge whether they are mathematically correct or faithful to the original. The human sees only AI reports and mechanical check results: whether Lean accepted each theorem and its axiom list. Based on them, the human makes the decisions in (P4).

Instead, another AI checked whether the definitions and the theorem statements are faithful to the original. This step was part of the research plan from the beginning. The construction and proof work used Claude Code, and this check used Codex CLI.

We followed the protocol in this sense throughout the research period. Unless a human action is stated explicitly, the work of that period (construction, search, formalization, proofs, and checks) was carried out by AI or by programs it ran. The human checked the content only when writing this note. The results had already been obtained under the protocol above.

By (P3), the meaning of a theorem depends on the definitions written in Lean. Lean accepts a proof even if a definition is wrong. Therefore, what needs checking is not the proofs but whether the definitions and the theorem statements correctly express the intended claims. For this reason, we summarize the main definitions in Section 2.2 and state the main theorems in Section 3.1.

The kernel guarantees the correctness of the proofs, and the check in (P2) guarantees the axioms, so the human does not need to check either.

However, the formulas in this note were transcribed from the Lean text into mathematical notation with AI assistance. Strictly, the objects to check are the Lean definitions and theorem statements to which they point. We also trust the definitions in Mathlib [5] that they use (such as computability) and the Lean system itself. The claim of this note is "we reached this far under this protocol." The human does not guarantee the faithfulness of the formalization. This correspondence has not been checked mechanically. The checks performed by AI are also based on the same inputs, so they do not guarantee it. An error in the formalization may therefore make some or all of the claims of this note fail.

2.2 Formalization

The AI represented a Turing machine as a finite list of quintuples $[q, s, t, D, q']$, where q is the source state, s the read symbol, t the written symbol, D the move, and q' the target state. States and tape symbols are represented by natural numbers. The directions L, R, and Z mean moving left, moving right, and staying at the same position, respectively. The AI formalized determinism and reversibility as predicates Deterministic and Reversible on rule tables. An instantaneous description (ID) consists of a state, a head position in $\mathbb{Z}$, and a two-way infinite tape indexed by $\mathbb{Z}$; the symbol 0 is the blank. An ID is finite if it has finitely many non-blank cells. The AI represented a finite ID by a code in $\mathit{Code} = \mathbb{N} \times \mathrm{List}\,\mathbb{N} \times \mathrm{List}\,\mathbb{N}$, where $\mathrm{List}\,\mathbb{N}$ denotes the type of finite lists of natural numbers. For $p \in \mathbb{Z}$ and a code (q, l, r), $\mathrm{toID}(p, (q, l, r))$ is the ID with state q and head position p that puts r on the cells from p to the right and l on the cells to the left of p, nearest to the head first, and the blank elsewhere. For an ID α, $\alpha.\mathit{pos}$ denotes its head position. For a rule table T and an ID α, $\mathrm{IsIDOf}(T, \alpha)$ holds if α has finitely many non-blank cells, its state occurs in T, and every cell holds the blank or a symbol that occurs in T. The enumeration $\mathrm{tm}(i)$ of TMs, for $i \in \mathbb{N}$, decodes i into a list of quintuples by the standard bijection with the natural numbers in Mathlib; if the result is not deterministic, it is the empty list. For a rule table T, $\mathrm{NStates}(T)$ is the number of states that occur as source states in the table, and $\mathrm{NSyms}(T)$ is the number of symbols that occur in the table, together with the blank 0. A halting state is not counted as a state. Section 3.3 relates this way of counting to Morita's convention. An active state is a state that occurs as a source state in the table. For a rule table T and an ID α, $\mathrm{Halts}_T(\alpha)$ means that T, starting from α, reaches an ID to which no transition applies after finitely many steps, possibly zero.

For universality, we use two definitions. The strong definition IsUTMStrict corresponds to Definition 7.1 of Morita's book [2], which includes computable encoding and decoding. For a rule table U, $\mathrm{IsUTMStrict}(U)$ holds if there exist an encoding $c : \mathbb{N} \times \mathit{Code} \to \mathit{Code}$ and a decoding $d : \mathit{Code} \to \mathrm{Option}\ \mathit{Code}$, computable in the sense of Mathlib, such that the following three conditions hold for all $i \in \mathbb{N}$ and codes a with $\mathrm{IsIDOf}(\mathrm{tm}(i), \mathrm{toID}(0, a))$. Here $\mathrm{Option}\ \mathit{Code}$ consists of values $\mathrm{some}(b)$ for codes b and a value none indicating that no code is returned. (1) $\mathrm{IsIDOf}(U, \mathrm{toID}(0, c(i, a)))$. (2) If $\mathrm{tm}(i)$ reaches a halting ID β from $\mathrm{toID}(0, a)$, then U reaches a halting ID γ from $\mathrm{toID}(0, c(i, a))$, γ has a code (a g with $\mathrm{toID}(\gamma.\mathit{pos}, g) = \gamma$), and for every such g there is a b with $d(g) = \mathrm{some}(b)$ and $\mathrm{toID}(\beta.\mathit{pos}, b) = \beta$. (3) If $\mathrm{tm}(i)$ does not halt from $\mathrm{toID}(0, a)$, then U does not halt from $\mathrm{toID}(0, c(i, a))$ either. The weak definition $\mathrm{IsUTM}(U)$, used for the nonexistence proofs, drops (1) and weakens "every code" in (2) to "some code". The strong definition implies the weak one. If two deterministic tables both satisfy IsUTMStrict and the same input is given to each through its own encoding, then they agree on halting, and if they halt, the decoded halting IDs agree. Any two of the witnesses of the 13 points agree in this sense. We do not claim that the output codes, the runs, or the numbers of steps agree. The AI formalized the simulation of an arbitrary cyclic tag system with halting condition (CTSH) as a relation separate from universality.

All theorems reported here as proved in Lean have been verified in Lean 4 with Mathlib and

meet the acceptance condition (P2). The Lean formalization and the full version, which includes construction details, proof outlines, and a comparison with prior work, are publicly available in the same repository at `https://github.com/yokoyama-lab/small-machine-zoo`.

2.3 Construction and search

A point (m, n) dominates (m', n') if $m \leq m'$, $n \leq n'$, and at least one of the two inequalities is strict. We used Morita's nine points as the baseline, and we counted a point as an improvement if none of them dominates it.

The human gave the direction: to make smaller URTMs by converting known URTMs. The human also chose the conversions to improve: the conversions to four states [2, Thm. 5.5], to three states [2, Thm. 5.6], and to two symbols [2, Thm. 5.4] in Morita's book. Later, the human directed the AI to survey the methods by which new points were obtained for standard UTMs by conversion or by revising tables, and to look for methods that apply to URTMs. The AI produced all the ideas for the improvements. There are five ideas. (1) Numbering that makes the marks unnecessary, with counting in the reverse direction. (2) Removal of counter symbols that do not occur. (3) Block decomposition and merging of roles. (4) Optimization of the codes in the conversion to two symbols. (5) Improvement of the conversion to three states. Kudlek and Rogozhin merged the blank with another symbol in a standard UTM [9]. The AI found this method in the survey above and applied it to the converted URTM tables. The AI also produced the rule tables, the solver models, and the Lean proofs. The human did not design the rule tables.

3 Results

3.1 Main theorems

We state the main theorems that the AI proved in Lean. The definitions are in Section 2.2.

Existence. For each of the 13 non-dominated points (m, n) in Figure 1,

$$\exists U.\ \mathrm{Deterministic}(U) \wedge \mathrm{Reversible}(U) \wedge \mathrm{NStates}(U) = m \wedge \mathrm{NSyms}(U) = n \wedge \mathrm{IsUTMStrict}(U).$$

At most two states. If U is deterministic and reversible and satisfies $\mathrm{NStates}(U) \leq 2$, then $\neg\mathrm{IsUTM}(U)$.

One symbol. If U is deterministic and $\mathrm{NSyms}(U) \leq 1$, then $\neg\mathrm{IsUTM}(U)$.

Three states and two symbols. If U is deterministic and reversible and there are distinct q_0, q_1, q_2 and $\sigma \neq 0$ such that every quintuple leaves one of q_0, q_1, q_2 and reads and writes 0 or σ, then $\neg\mathrm{IsUTMStrict}(U)$.

One left-moving quintuple. If U is deterministic and reversible, has at most one quintuple with direction L, and for every quintuple ℓ with direction L no quintuple with direction Z has the source state of ℓ as its target, then $\neg\mathrm{IsUTM}(U)$. The same holds for R.

Halting of two-state RTMs. There is a computable Boolean-valued function f such that, for every choice of source states q_0, q_1 and a halting state q_f, every deterministic and reversible table T whose source states are q_0 and q_1 and whose targets are q_0, q_1, or q_f, and all positions $p \in \mathbb{Z}$ and codes $z \in \mathit{Code}$, $f(T, p, z) = \mathrm{true} \Leftrightarrow \mathrm{Halts}_T(\mathrm{toID}(p, z))$. Here q_f is not a source state of T.

3.2 New URTMs

Figure 1 shows the results. For each new point, the original machine is shown to the right of an arrow, as in $(4, 42) \leftarrow (13, 7)$. Six of Morita's nine points are machines designed by Morita ($(17, 5)$ by Morita and Yamaguchi). The other three, $(4, 168)$, $(138, 2)$, and $(3, 36654)$, were obtained from them by conversion. In this research, the AI improved Morita's conversions and applied them to the six original machines, adding symbol merging for some of them. The AI obtained seven points: six with 3 to 8 states and one with 2 symbols. During the research, the AI first obtained $(4, 46)$, $(5, 45)$, $(6, 37)$, $(7, 32)$, $(8, 28)$, and $(9, 27)$ by block decomposition and merging of roles alone, and it proved their universality in Lean. The five points with symbol merging, $(4, 42)$, $(5, 39)$, $(6, 30)$, $(7, 28)$, and $(8, 21)$, dominate all six of them, so the figure does not show them. The dash-dotted line in the figure bounds the region where UTMs are known not to exist [8]. This region consists of one-state machines, two-state two-symbol and three-state two-symbol machines (under a definition that reserves one transition rule for halting), and one-symbol machines. These results concern TMs that are not necessarily reversible, and they are proved on paper. Their treatment of halting and zero-shift moves differs from our definitions, so we do not claim that they apply directly to URTMs in our sense. The green dashed line bounds the region where we proved in Lean that no URTM with at most two states exists (Section 3.4). The green cross marks the nonexistence of three-state two-symbol URTMs (Section 3.5). Both are proved in Lean in the sense of IsUTMStrict, the universality used for the 13 points.

For the five symbol-merging points, the AI chose the tables with solvers and simulated annealing. The coloring assigns a resulting symbol to each original symbol, so symbols assigned the same color are merged. We claim neither minimality of the number of colors nor minimality over all choices of blocks. The point $(85, 2)$ was found by an automated search modeled on Karpathy's autoresearch [7], in which the human did not intervene after starting it. Six of Morita's nine points (from $(10, 8)$ to $(32, 3)$) remain non-dominated, so there are 13 non-dominated points.

The AI proved the following properties in Lean.

- The concrete rule tables of the 13 points and of Morita's nine points (16 tables without duplicates) are deterministic and reversible. The 11 tables other than the five symbol-merging tables, and the five block tables before merging, simulate an arbitrary CTSH (halting, non-halting, and halting IDs correspond).

- All 13 points are universal in the sense of Morita's Definition 7.1 as formalized in Lean. That is, a computable encoding maps a finite ID of an arbitrary TM to an ID of the machine of each point, and halting and non-halting agree. Moreover, a computable decoding maps the halting ID back to the halting ID of the original TM.

The AI first proved the case of $(10, 8)$. The human then directed the AI to use a general theorem combining CTSH simulation with the input encoding and output extraction for each of the remaining points. The AI proved the theorem and applied it to those points. For the five symbol-merging points, the AI proved and used the following general theorem in Lean. Suppose that a table U is universal in the sense of Definition 7.1 and that every halting ID reached from an encoded input is not in a source state. Assume that there is a single finite set of symbols $F \subseteq \mathbb{N}$ containing all tape symbols in every such halting ID. Let $h, \rho : \mathbb{N} \to \mathbb{N}$ be primitive recursive maps. The map h assigns a new symbol to each original symbol, while ρ recovers the original symbols in F. Assume that $h(0) = \rho(0) = 0$ and that $\rho(h(x)) = x$ for every $x \in F$. Replace the read and written symbols of every quintuple of U by their images under h. If the resulting table is deterministic, then it is also universal. Morita distributes description files containing rule tables and test runs for nine machines. The AI reproduced the published halting step counts in Lean, except for one count corrected by recomputation. The witnesses are the tables in Morita's distributed files and the tables obtained from them by our conversions. For the 11 points other than $(85, 2)$ and $(3, 600)$, the distributed tables represent the entries marked as halting in the

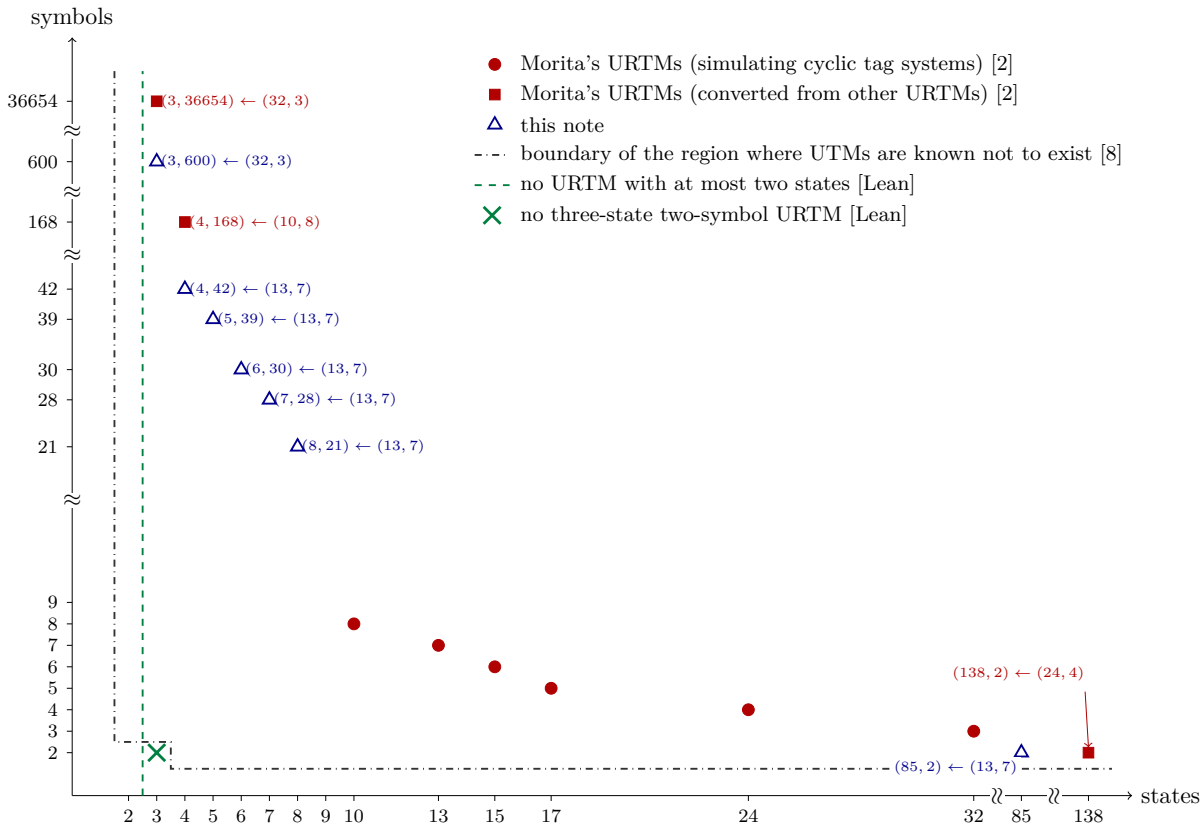


Figure 1: Numbers of states and symbols of URTMs

book by two zero-shift quintuples leading to halting states that are not counted. For all 11 points, the AI also proved universality in the sense of Definition 7.1 for the tables after removing these two quintuples. The resulting tables are those in the book or, for the five symbol-merging points, their analogues. Removing the two quintuples does not change the numbers of states and symbols. The decoder first maps a halting ID back to the halting state and then applies the original decoding.

3.3 Counting states and symbols

Morita does not count halting states in the m of URTM(m, n). Morita states that the set of final states is not specified in the problem of reducing the number of states, and the state sets of the URTMs in Chapter 7 contain no halting states. In Chapter 11, Morita states that halting states are not included in URTM$(17, 5)$, so two states are added [2]. The distributed files also state explicitly that the halting states halt and null are not counted.

In Lean, NStates is the number of source states, and NSyms is the number of symbols that occur in the table, together with the blank (Section 2.2). The two ways of counting can differ in three cases: states that occur only as targets, halting states, and symbols that do not occur in the table.

For the witness tables of all 13 points described in Section 3.2, the AI proved in Lean that every target is a source state. For $(85, 2)$ and $(3, 600)$, the tables in the form of the distributed files satisfy this condition themselves. Therefore, these witnesses have neither states that occur only as targets nor halting states. Even under a convention that also counts target states, the number of states stays m.

For the symbols, the AI proved in Lean that the blank occurs in each witness and that every symbol is both read and written. This agrees with an independent count performed outside Lean

Table 1: Independent count of states and symbols in the witness tables

Table	Quint.	Sources	Target-only	Symbols	Zero-shifts
Morita's 6 points, distributed	59–84	m	2	n	2
same, tables in the book	57–82	m	0	n	0
5 symbol-merging points, distributed	143–162	m	2	n	2
same, without 2 quintuples	141–160	m	0	n	0
$(85, 2)$	153	85	0	2	0
$(3, 600)$	1,572	3	0	600	0

(Table 1). Moreover, the AI defined Morita's way of counting in Lean (it also counts target states and counts only the symbols that occur in the table). The AI proved that the witnesses of the 13 points have exactly (m, n) under this way of counting as well.

For the 11 distributed tables with the additional halting quintuples described in Section 3.2, the AI proved in Lean that counting target states gives $m + 2$ states.

3.4 Nonexistence of URTMs with at most two states

The AI proved the following in Lean. For every RTM with at most two states, regardless of its number of symbols, the halting problem from finite IDs is decidable (the one-state case is reduced to the two-state case). Moreover, the AI showed in Lean that a single computable procedure, which also takes the table T as input, decides halting for every two-state RTM (uniform decision). Therefore, no URTM with at most two states exists in the sense of Definition 7.1 as formalized in Lean (the green dashed line in Figure 1). If this formalization correctly expresses Morita's definition, this answers negatively a question that Morita [2] left open.

Under the same assumption, the decidability of the halting problem of two-state RTMs also answers negatively the question in Morita's 2023 survey quoted in the introduction. The AI showed in Lean that there is a TM with an undecidable halting problem that no two-state RTM simulates.

Morita's RTMs do not count halting states and also allow transitions that do not move the head (zero-shifts). The results above hold under this definition. When there are several halting targets, removing the rules whose target is not an active state does not change whether the machine halts. If the original table halts by executing a removed rule, the table without it halts one step earlier. Removing rules preserves determinism and reversibility, so the two-state theorem applies to the resulting table. The AI proved this reduction in Lean for tables with at most two kinds of source states. The AI also showed in Lean that a table whose only symbol is the blank is not universal, even if it is not reversible. The tape never changes, so a halting run halts within a bounded number of steps.

The AI also proved in Lean a reversible version of Pavlotskaya's Theorem 4 [10], which states that the halting problem is decidable for two-symbol TMs with only one instruction that moves left. A deterministic and reversible table with at most one quintuple that moves left is not universal, regardless of its numbers of states and symbols, provided that no zero-shift quintuple enters the source state of that quintuple (the same holds for quintuples that move right). By reversibility, the only quintuple that enters the target of the unique left-moving quintuple is that quintuple itself.

3.5 Nonexistence of three-state two-symbol URTMs

The AI proved in Lean that no three-state two-symbol RTM is universal under a formalization of Definition 7.1 that uses Morita's string IDs. This formalization also requires that the encoded values use only the blank and one symbol of the machine. For deterministic two-symbol tables, the only symbols that occur are the blank and one nonblank symbol, so the AI proved that

IsUTMStrict implies this form. Combining this transfer with the nonexistence result, the AI also proved that no such RTM is universal in the sense of IsUTMStrict, the universality used for the 13 points. The theorems cover deterministic and reversible tables with at most three source states whose symbols are the blank and one other symbol. Halting states and zero-shifts are allowed, as in Morita's definition. The AI used prior work on irreversible TMs as a guide for the proof. The author has not checked this use against the original sources. Applying the same transfer to the existence theorem for $(85, 2)$, the AI showed that at least one machine satisfies this form. This proof uses two Lean environments and relies on the copied definition being exactly the same in both.

4 Conclusion

Under the stated protocol, the AI constructed seven URTMs not dominated by Morita's nine points and proved the universality of all 13 non-dominated points in Lean. It also proved the nonexistence of URTMs with at most two states and of three-state, two-symbol URTMs.

All these conclusions hold only if the Lean definitions and theorem statements correctly express the intended claims. This correspondence has not been checked mechanically, so an error in the formalization may make some or all of the conclusions fail.

Acknowledgments

We are grateful to the late Kenichi Morita, who introduced us to this interesting problem and encouraged us to take it on. This work was supported by JST CREST Grant Number JPMJCR24I4, Japan.

This manuscript was first drafted in Japanese through dialogue with generative AI and then translated into English using generative AI. The author subsequently revised the English wording. The author made the final decisions on the manuscript and takes responsibility for its content.

A Construction details and intermediate counts

This appendix brings together the construction and proof explanations that accompany the short note. The Lean definitions and theorem statements determine the exact hypotheses and conclusions. The explanations below do not constitute a human inspection of the formalization.

A.1 Four-state and block constructions

Let an original machine have m states and n symbols. Write m_+ and m_- for the numbers of states entered by right and left moves, respectively. Morita's four-state conversion uses $(2m+1)n$ symbols. Giving the two directions separate number ranges reduces the expression to $(m+1)n$. Counting in reverse while carrying a symbol to the left gives the upper bound $(\max(m_+, m_-)+1)n$. Splitting states into blocks, merging roles, and removing unused symbol–counter pairs gives $n + \sum_{i \in V} |X_i|$, where V is the set of counter values used and X_i is the set of original symbols used with value i. For the conversion from $(10, 8)$, the respective symbol counts are 168, 88, 48, and 46. The last value is the block construction before the later symbol-merging improvements.

The corresponding formal statements are collected in `LeanUniv.FinalFormulas`. The expression for reverse counting is an upper bound; the other formulas just listed are equalities for the stated conversion functions. These formulas concern construction families and do not give lower bounds over all universal machines.

For the final five symbol-merging points, the original machine is $(13, 7)$. The recipe files specify the blocks and counter choices, and the merge files specify classes of symbols. The generators reconstruct the tables, while Lean checks their finite properties and applies the general

symbol-merging theorem. Rebuilding these files reproduces the selected constructions; it does not repeat the optimization search or establish optimality.

A.2 Three-state conversion

For Morita's three-state conversion, let $M = m_+n + 2n + 2$ and $N = mn^2 + n^2 + 1$ be the sizes of the intermediate machine. The original symbol-count formula is $N(2M - 1)$. Sharing the up and down marks gives NM. A further construction assigns a separate counter limit to each symbol; its count is N plus the sum of these limits.

For the source $(32, 3)$, the successive recorded symbol counts are 36,654, 18,476, 1,820, 659, and 600. The 1,820-symbol intermediate table has only a size calculation: adequacy of its counter limits throughout every run was not proved. The 659-symbol table rebuilds the intermediate machine using one symbol per cell. Reflecting the source machine left to right gives the 600-symbol table. Simulation and universality of the final $(3, 600)$ witness are established by the corresponding Lean theorems.

A.3 Two-symbol conversion and automated search

For a conversion using k-bit codes, let P_r and P_w be the sums, over states, of the numbers of read-code and write-code prefixes of lengths $1, \ldots, k-1$. Let D_0 be the set of source states having a rule whose target is also a source state. The formalized state-count formula is $|D_0|+(k-1)m+P_r+P_w$, bounded above by $mk + P_r + P_w$.

From $(13, 7)$, the recorded state counts are 169 using Morita's code, 149 with a search-selected code, 135 after selecting written values and path types, 92 after state merging, and 85 after the broader automated search. The solver that selected the intermediate tables is not itself a verified Lean conversion function. Its concrete output tables are checked in Lean; the proof for the merged table also establishes the required relationship to its pre-merge table. The relevant intermediate-value theorems are in `LeanUniv.FinalValues`.

The automated search was modeled on autoresearch [7]. An interactive AI session prepared the search program, fixed evaluator, and instructions at the human's request. After the human started a session, the agent edited the search program and the evaluator checked candidates for determinism, reversibility, and finite CTSH simulations. Only candidates subsequently proved in Lean counted as results. The search found $(6, 37)$, $(8, 28)$, $(9, 27)$, and $(85, 2)$; only the last remains non-dominated after the symbol-merging improvements. The repository includes the construction recipes needed to rebuild the final witnesses.

B Outline of the two-state decidability proof

The formal entry points are `halting_decidable_uniform` and `no_le_two_state_URTM`. This section summarizes the structure of the existing proof; the formal statements specify its scope.

First remove transitions into states with no outgoing transitions by the reduction described in the main text. Let $[a, b]$ contain the initial head position and all initially non-blank cells, and put $w = b - a + 1$. Let Σ contain the blank and every symbol read or written by the table, and let $n = |\Sigma|$. Consider an initial ID whose tape symbols lie in Σ.

1. In a reversible table, transitions entering the same state move in the same direction. If a two-state table has both left and right moves, one state is entered only from the left and the other only from the right; there is no remaining state for a zero-shift target. The state therefore records the direction of the last move. The transition function can be represented by a partial injection on direction–symbol pairs.

2. Consider a half-line to the right of a fixed boundary. Every entry has the right-moving state and every exit the left-moving state. While the head is outside the half-line, its contents

do not change. The outcome of the next entry therefore depends on its initial contents and on the number of earlier entries, rather than on the outside tape. The only information passed back to the outside is the return itself.

3. For initially blank nested half-lines, their local computations are translations of one another. In a halting computation, each visited deeper half-line has fewer entries than its predecessor. Otherwise the computation repeats translated to the right and never halts. This bounds the maximum depth by the number of entries at the outer boundary.

4. The sequence of visits to a cell follows a finite local clock on direction–symbol pairs. There are $2n$ such pairs. This bounds by $2n+1$ the number of entries into a blank half-line if the computation halts inside it. The left-hand case follows by reflection.

5. Replace excursions outside $[a, b]$ by immediate returns at its ends. The resulting dynamics within the window is finite. In the existing proof, $Z = n^w \cdot 3w$ bounds the window states, and $N = Z(2n+3)+1$ bounds the relevant entries into each outside half-line. Combining this with the depth bound confines a halting run to $[a-N, b+N]$.

6. That interval contains finitely many IDs over the finite alphabet. A halting run of a deterministic machine cannot repeat an ID. Thus a computable bound on the number of steps suffices to decide halting by bounded simulation. The bound need not be practical.

If only one movement direction occurs, once the head leaves the initial window it repeatedly reads blanks. Repetition of a state gives a translated repetition of the run. A halting run can therefore be bounded in this case as well. The general nonexistence theorem combines the computability of halting for these tables with Mathlib's undecidability of the halting problem.

The first step is where reversibility restricts the two-state model. With three states, a half-line can receive different kinds of entries moving in the same direction; the two-state argument does not apply unchanged.

C Outline of the three-state two-symbol proof

The proof normalizes the tables and checks 359,829 normalized tables with the Lean kernel. It handles most tables by certificates constraining visits outside a window and by finite-state bounds. Other cases reduce to bounded halting for a two-state machine after removing a state, or require a separate argument about their behavior.

Counts of equivalence classes up to renaming and left–right reflection were computed outside Lean: there are 26,861 such classes in the recorded enumeration, and the last ten classes contain 120 tables. The AI supplied arguments for these final classes, proving that a halting run stays within a bounded neighborhood of the initial window.

The resulting theorem is `URTM.Meta.no_three_two_URTM_all`, followed by the transfer to `no_three_two_URTMStrict_all`. The original target condition is the string-ID form of universality with the stated restriction on encoded symbols. The transfer from `IsUTMStrict` is essential: a proof of nonexistence for a different universality definition alone would not imply the claim made in the note. The finite enumeration files and all imported proof dependencies are included in the repository.

D Comparison with prior work and verification scope

Morita's nine machines provide the baseline [2]. Six of these points remain on the frontier together with the seven new points. The work improves conversion and symbol-assignment choices; it does not prove that the resulting state–symbol pairs are globally smallest. Kudlek and Rogozhin's

merging of a blank and another symbol in an ordinary UTM [9] guided the symbol-merging stage. The general theorem here makes the preservation conditions explicit for the formalized universality predicate, including recovery of the symbols on halting tapes.

The lower-bound region quoted for standard UTMs comes from the survey by Woods and Neary [8]. Its conventions for halting and zero-shifts differ from the present model. The new nonexistence claims are therefore established directly in the stated reversible model rather than inferred solely from that comparison. The AI also used prior work on irreversible machines to guide the three-state proof; the author has not checked that correspondence against the original sources.

The supplied generators read Morita's original test configurations from the machine descriptions attached to his 2015 slides. For eight machines, Lean reproduces 44 published halting runs, with the count 1,073 in one $(13, 7)$ comment corrected to 1,173. For $(3, 36654)$, the exact step counts are 471,798, 2,348,492, and 57,220. The large table is checked through a lookup function proved to agree with it and certificates for successive run segments. These finite examples support comparison of the operational model with the supplied data, while the general simulation and universality theorems cover arbitrary encoded inputs.

The two proof environments reuse the specification by copying it. The repository checks that the copies of `Spec.lean` are byte-identical and that the copies of `Strict.lean` differ only in their imports. Each environment is built separately and its imported declarations are audited for nonstandard axioms. These checks establish properties of the formal artifacts; they do not establish the faithfulness of the formalization to the intended mathematical claims.

References

[1] C. H. Bennett. Logical reversibility of computation. *IBM Journal of Research and Development*, 17(6):525–532, 1973. https://doi.org/10.1147/rd.176.0525.

[2] K. Morita. *Theory of Reversible Computing*. Springer, 2017. https://doi.org/10.1007/978-4-431-56606-9.

[3] K. Morita. Making reversible computing machines in a reversible cellular space. *Bulletin of the EATCS*, no. 140, 2023. https://bulletin.eatcs.org/index.php/beatcs/article/view/760.

[4] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, Lecture Notes in Computer Science, pages 625–635. Springer, 2021. https://doi.org/10.1007/978-3-030-79876-5_37.

[5] The mathlib Community. The Lean mathematical library. In *Proc. 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381, 2020. https://doi.org/10.1145/3372885.3373824.

[6] L. Murphy, K. Yang, J. Sun, Z. Li, A. Anandkumar, and X. Si. Autoformalizing Euclidean geometry. In *Proc. 41st International Conference on Machine Learning (ICML 2024)*, PMLR 235, pages 36847–36893, 2024.

[7] A. Karpathy. autoresearch. https://github.com/karpathy/autoresearch, 2026.

[8] D. Woods and T. Neary. The complexity of small universal Turing machines: A survey. *Theoretical Computer Science*, 410(4–5):443–450, 2009. https://doi.org/10.1016/j.tcs.2008.09.051.

[9] M. Kudlek and Y. Rogozhin. A universal Turing machine with 3 states and 9 symbols. In *Developments in Language Theory*, Lecture Notes in Computer Science 2295, pages 311–318, Springer, 2002. https://doi.org/10.1007/3-540-46011-X_27.

[10] L. M. Pavlotskaya. Solvability of the halting problem for certain classes of Turing machines. *Mathematical Notes of the Academy of Sciences of the USSR*, 13(6):537–541, 1973. https://doi.org/10.1007/BF01163965.